

AvalanCH

Digital Twin of the Alps

Rémy Fayet, Kevin Garnier, Laura Kubler, Kevin Peymani

10/04/2026

Motivation



How dangerous is an avalanche ?



Multiple stakes:

- ▶ People
- ▶ Buildings
- ▶ Roads
- ▶ Train tracks
- ▶ ...

Figure 1: House destroyed by the passage of an avalanche in Idaho

Chances of survival

Detection has to be quick ! Realtime !

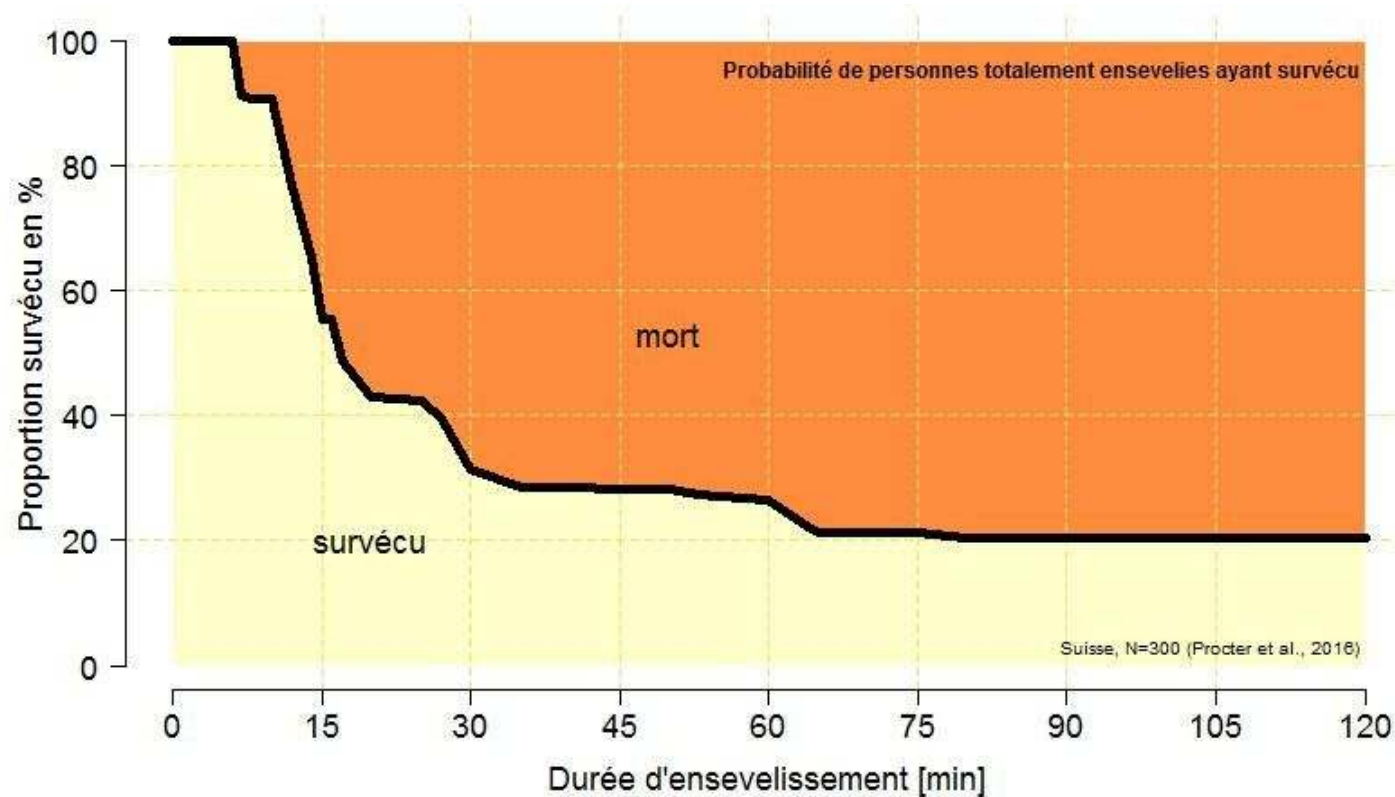


Figure 2: Survival rate of a skier in an avalanche over time (based on 300 skiers)

Can we prevent avalanches?

- ▶ Avalanches are mostly caused by skiers (backcountry skiers)
- ▶ We can't predict human behavior (yet?)



Figure 3: Backcountry skier triggering an avalanche

But...

- ▶ We can calculate the probability of an avalanche occurring, given **weather data**, **snow profile** and a **zone of release**.



Figure 4: Swiss weather station measuring snow depth

Is that enough to ensure safety?

- ▶ In Switzerland (as in many countries), an avalanche risk daily report is available
- ▶ Unfortunately, casualties still happen



Avalanches have claimed 15 lives in Switzerland this winter

Mar 31, 2026 • This winter significantly more people than usual have been buried by avalanches in the Swiss mountains.

Figure 5: Article on swiss avalanche casualties this winter

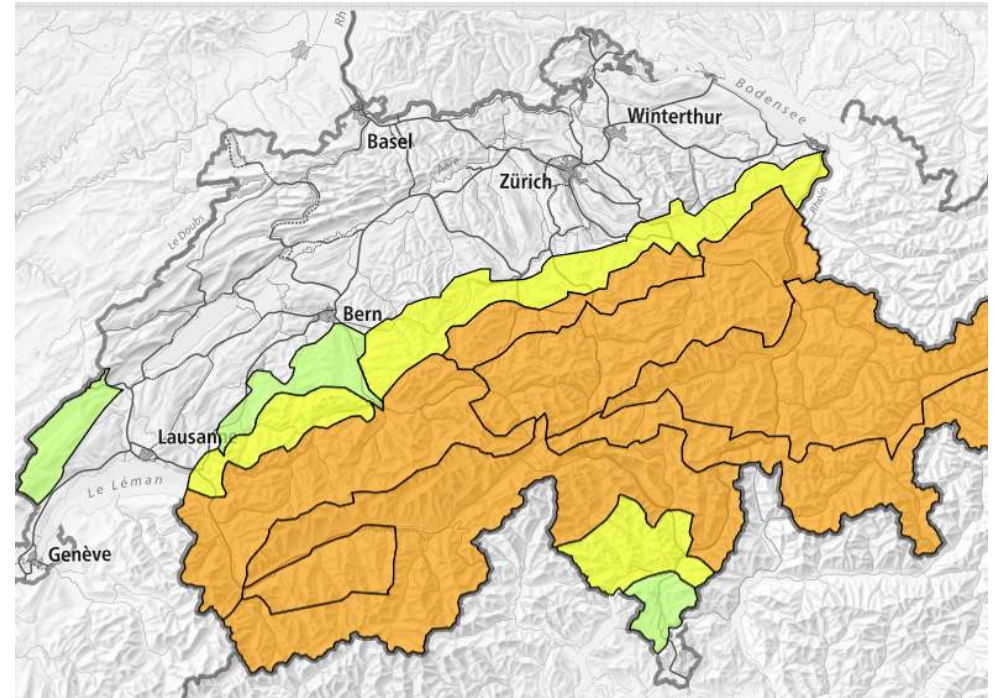


Figure 6: Avalanche risk map, Switzerland

Value provided



Why digital twins are the answer for avalanche prevention

Avalanche prevention requires gathering various domain experts :

- ▶ weather models for prevision
- ▶ snow height prevision using distributed autonomous stations
- ▶ infrastructure intelligence to qualify the risks

Why digital twins are the answer for avalanche prevention

Avalanche prevention requires gathering various domain experts :

- ▶ weather models for prevision
- ▶ snow height prevision using distributed autonomous stations
- ▶ infrastructure intelligence to qualify the risks

DTs provide a solution:

- ▶ **Continuous risk monitoring** : with sensor arrays, avalanches could be instantly detected and reflected in the DT
- ▶ **Rapid alerting and dispatch** : Rescue teams could be alerted as soon as snow starts moving
- ▶ **Simulation of scenarios** : Predict future avalanche risks, for crisis prevention infrastructure projects, etc..

Why Switzerland ?

- ▶ Switzerland has snowy mountains
- ▶ Most geographic and weather data is publicly available through REST APIs



Figure 7: Matterhorn in Zermatt, Switzerland

Ground-based measurements

DATASET / CONTENT	DOWNLOAD	STAC BROWSER	STAC API
Automatic weather stations	File	Collection	Collection
Temperature, precipitation, wind, sunshine, humidity, radiation and pressure – every 10 minutes, hourly, daily, monthly and yearly Documentation			
Automatic precipitation stations	File	Collection	Collection
Precipitation – every 10 minutes, hourly, daily, monthly and yearly Documentation			
Automatic tower stations	File	Collection	Collection
Temperature, wind, sunshine, humidity and radiation – every 10 minutes, hourly, daily, monthly and yearly Documentation			
Manual precipitation stations	File	Collection	Collection
Precipitation and snow – daily, monthly and yearly Documentation			
Totaliser precipitation stations	File	Collection	Collection

Figure 8: Swiss ground-based weather measurements database

Our DT



Dataflow

We have several data sources

SwissAlti3D



SwissIMAGE



IMIS network



SLF weather



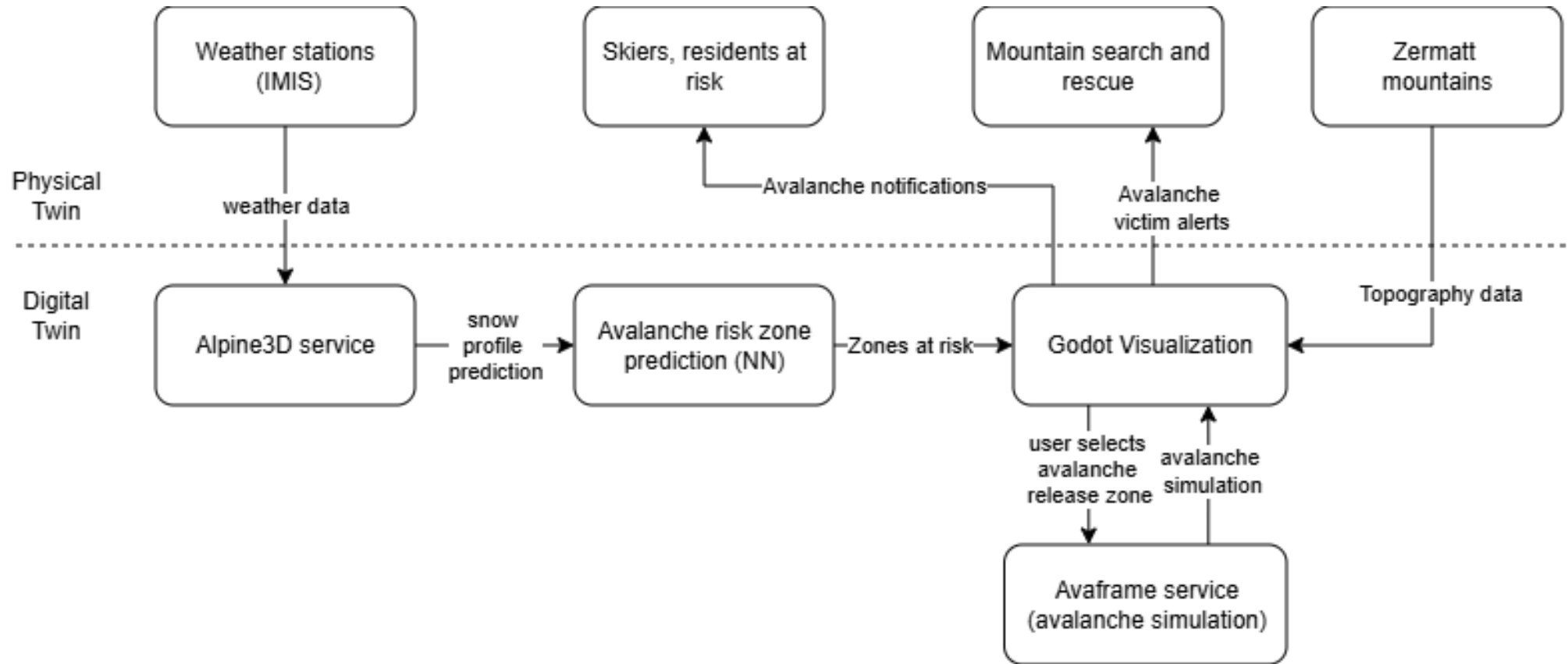
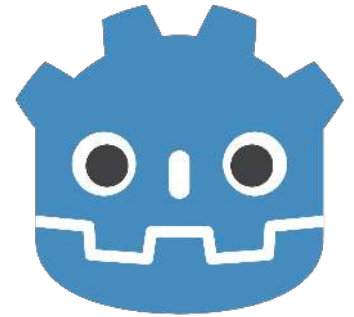
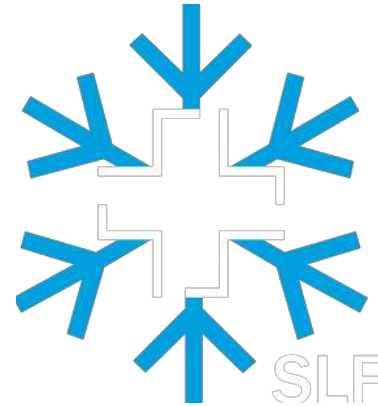


Figure 9: Diagram of the AvalanCH-DT



Alpine3D



- ▶ Everything is huge (Switzerland)
- ▶ Everything is slow (Slow evolution, timescale ~days)

Therefore:

- ▶ we can't simply use RabbitMQ (sharing using docker volumes)
- ▶ we can't quickly iterate using AGILE for example

DTInsight



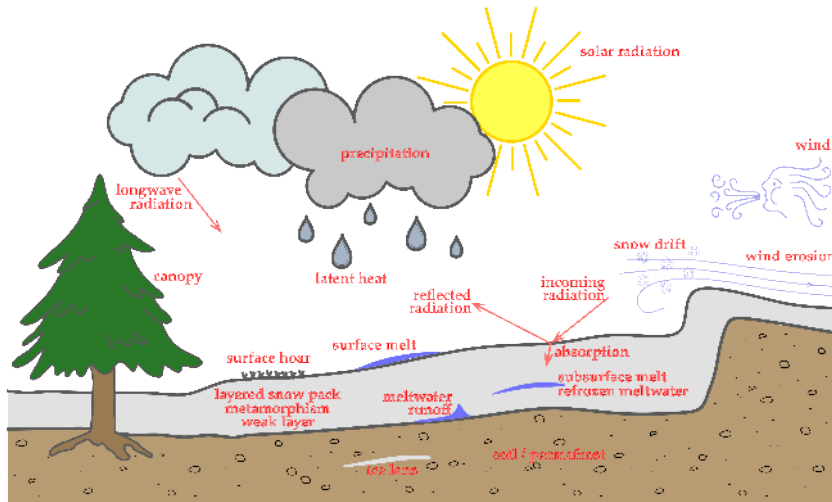
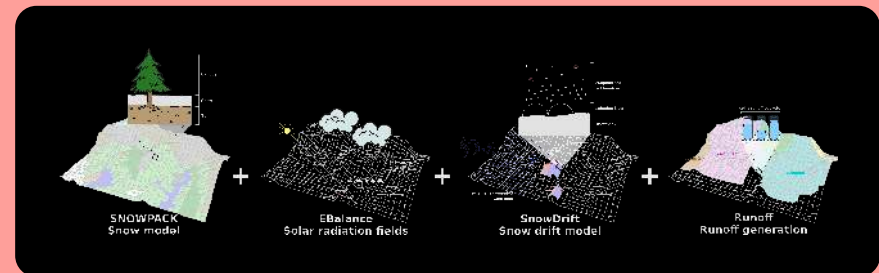


Figure 10: Alpine3D physical processes

- ▶ spatially distributed (surface)
- ▶ three dimensional (atmospheric) model
- ▶ analyzing and predicting dynamics of snow-dominated surface processes in mountains
- ▶ includes the snow cover model **Snowpack**



How do we get the data ?

```
GET /public/api/imis/measurements
```

Response samples

200

Content type
application/json

Copy Expand all Collapse all

```
[
  - {
    "station_code": "string",
    "measure_date": "2019-08-24T14:15:22Z",
    "HS": 0,
    "TA_30MIN_MEAN": 0,
    "RH_30MIN_MEAN": 0,
    "TSS_30MIN_MEAN": 0,
    "TS0_30MIN_MEAN": 0,
    "TS25_30MIN_MEAN": 0,
    "TS50_30MIN_MEAN": 0,
    "TS100_30MIN_MEAN": 0,
    "RSWR_30MIN_MEAN": 0,
    "VW_30MIN_MEAN": 0,
    "VW_30MIN_MAX": 0,
    "DW_30MIN_MEAN": 0,
    "DW_30MIN_SD": 0
  }
]
```



```
def write_alpine3d_smet(station_code: str, hourly_df: pd.DataFrame,
                        metadata: dict) -> Path:
    INPUT_DIR.mkdir(parents=True, exist_ok=True)
    output_path = INPUT_DIR / f"{station_code}.smet"

    hs_series = hourly_df.get("HS", pd.Series(0.0,
                                              index=hourly_df.index)).astype(float) / 100.0

    forcing = pd.DataFrame(index=hourly_df.index)
    forcing["TA"] = hourly_df.get("TA_30MIN_MEAN", pd.Series(-1.0,
                                                             index=hourly_df.index)) + 273.15
    forcing["RH"] = hourly_df.get("RH_30MIN_MEAN", pd.Series(85.0,
                                                             index=hourly_df.index)) / 100.0
    forcing["VW"] = hourly_df.get("VW_30MIN_MEAN", pd.Series(2.0,
                                                             index=hourly_df.index))
    forcing["DW"] = hourly_df.get("DW_30MIN_MEAN", pd.Series(180.0,
                                                             index=hourly_df.index))
    forcing["ISWR"] = hourly_df.get("RSWR_30MIN_MEAN", pd.Series(150.0,
                                                                index=hourly_df.index))
    derived_ilwr = derive_ilwr_series(hourly_df)
    forcing["ILWR"] = hourly_df.get("ILWR_30MIN_MEAN",
                                    derived_ilwr).fillna(derived_ilwr)
    forcing["HS"] = hs_series
    forcing["TSG"] = hourly_df.get("TS0_30MIN_MEAN", pd.Series(-0.5,
                                                                index=hourly_df.index)) + 273.15
    forcing["PSUM"] = hourly_df.get("PSUM",
                                    derived_psum).fillna(derived_psum)
    forcing = forcing.ffmpeg().bfill().fillna(0.0)
```

How do we fix missing data ?

```
def derive_ilwr_from_components(ta_c, rh_pct, rswr_wm2):
    ta = np.asarray(ta_c, dtype=float)
    rh = np.clip(np.asarray(rh_pct, dtype=float), 1.0, 100.0)
    rswr = np.clip(np.asarray(rswr_wm2, dtype=float), 0.0, 1200.0)

    tk = ta + 273.15
    es_kpa = 0.611 * np.exp((17.27 * ta) / (ta + 237.3))
    ea_kpa = (rh / 100.0) * es_kpa

    eps_clear = 1.24 * np.power(np.maximum(ea_kpa, 1e-6) /
np.maximum(tk, 1.0), 1.0 / 7.0)
    eps_clear = np.clip(eps_clear, 0.62, 0.98)

    daylight_fraction = np.clip(rswr / 800.0, 0.0, 1.0)
    cloudiness = 1.0 - daylight_fraction
    eps_effective = np.clip(eps_clear + 0.25 * cloudiness, 0.65, 1.0)

    ilwr = eps_effective * STEFAN_BOLTZMANN * np.power(tk, 4)
    ilwr = np.clip(ilwr, 180.0, 420.0)

    if ilwr.size == 1:
        return float(ilwr.item())
    return ilwr
```

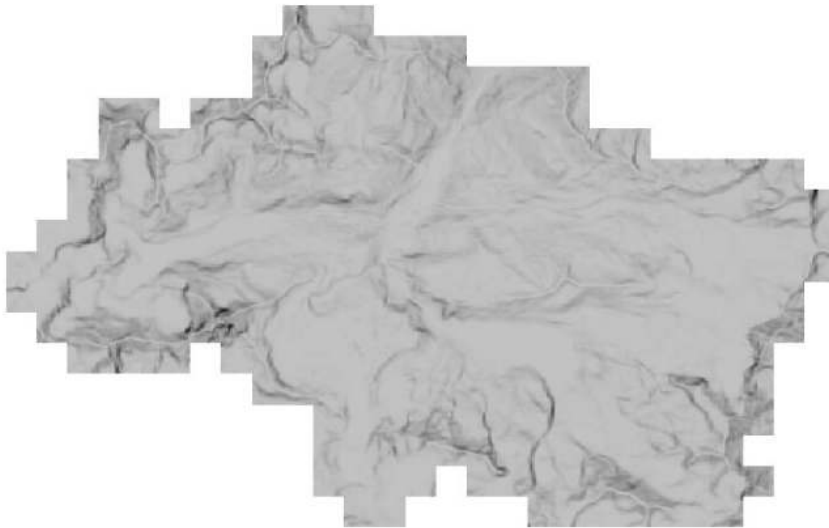
- ▶ Estimates the Incoming Longwave Radiation from existing data
- ▶ Using the The Stefan-Boltzmann Law:

$$ILWR = \varepsilon_{\text{eff}} \times \sigma \times T_K^4$$

- ▶ Different during the night (RSWR reflects how cloudy it is, 0 at night)

What does Alpine3D send back ?

- ▶ 2D grid of:
 - Snow Height: HS
 - Temperature Surface Ground (TSG)
 - Snow Water Equivalent (SWE) → the weight of mass of water contained in the snow

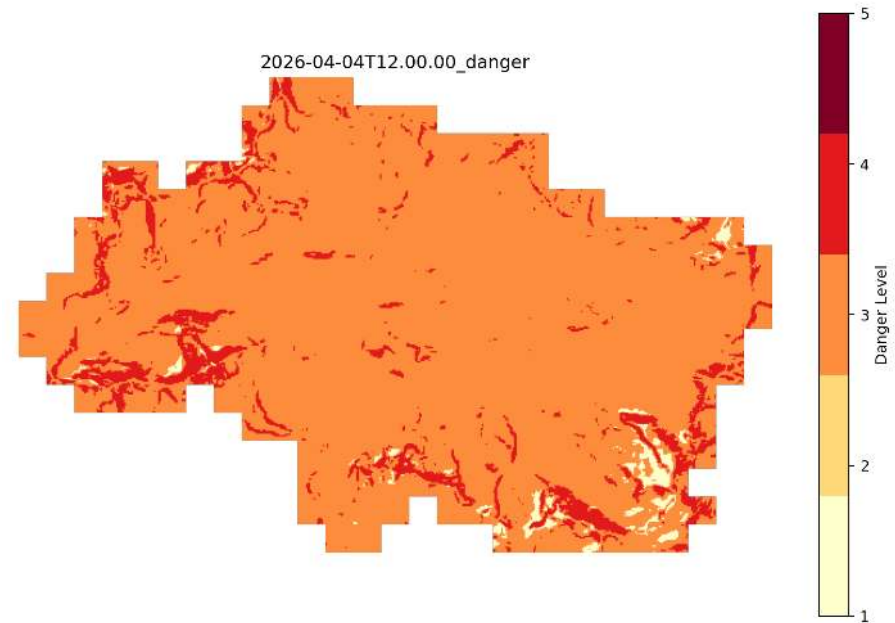
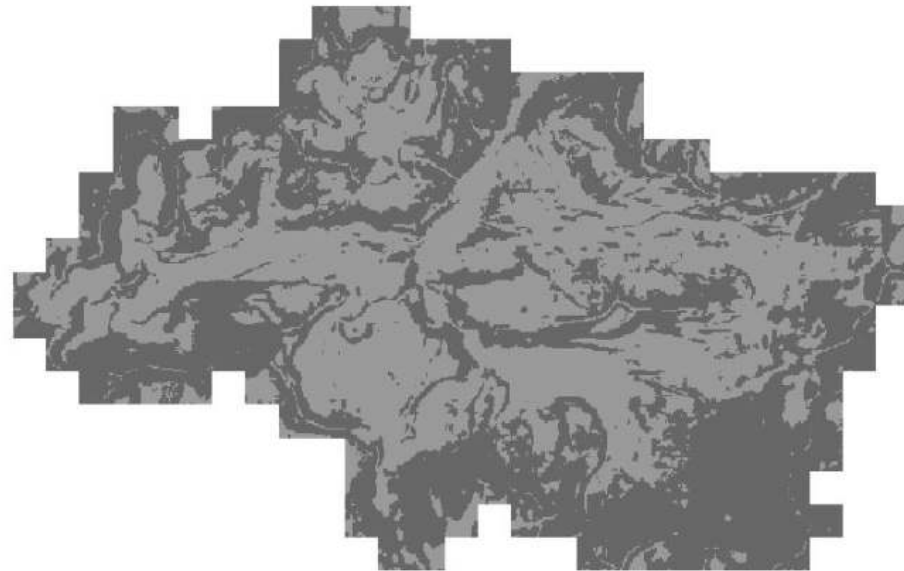


From this, how do
we predict
avalanche danger ?

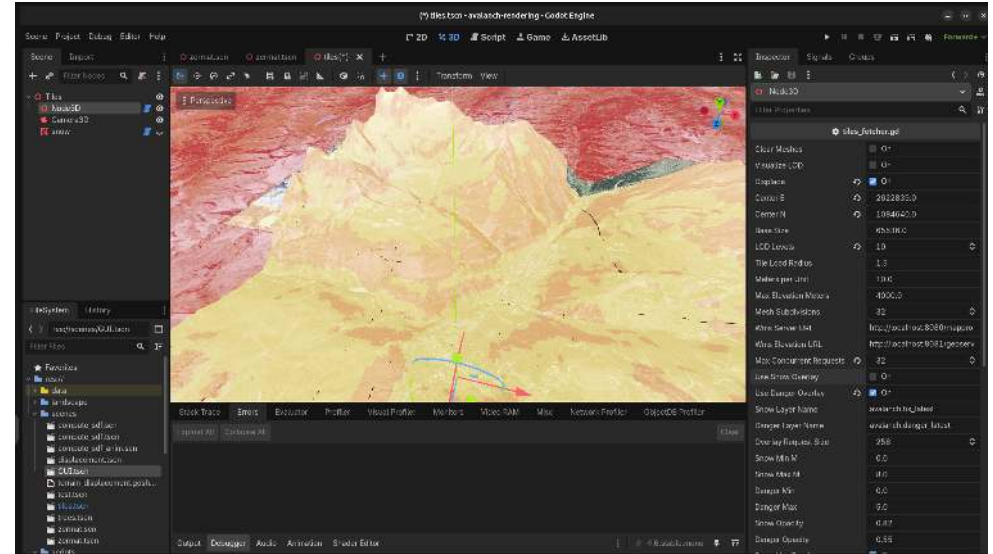
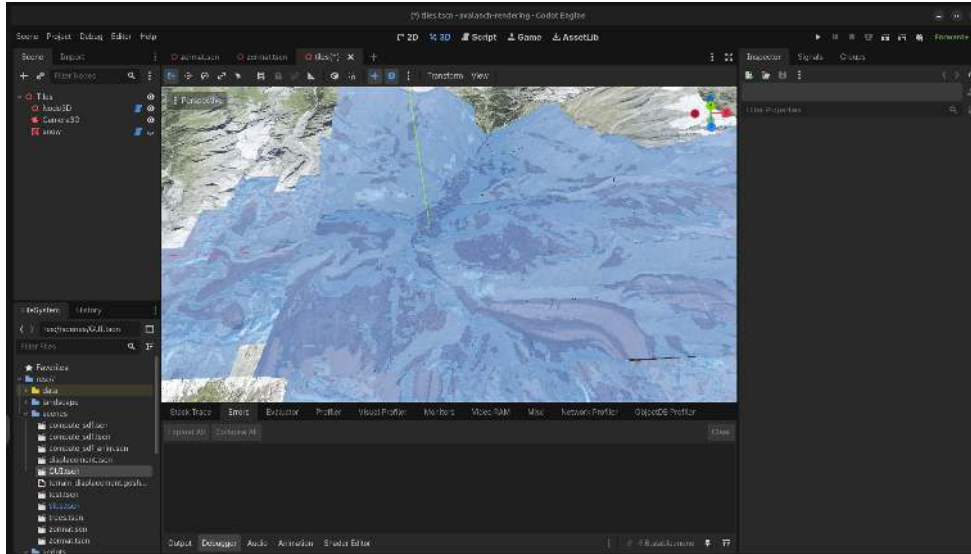
```
["HN24", "HN72_24", "HN24_7d",  
"Pen_depth", "min_ccl_pen",  
"wind_trans24",  
"wind_trans24_3d", "TA", "RH",  
"VW", "DW", "ISWR", "pAlbedo",  
"Qw", "MS_Snow", "SWE", "TS1",  
"TS2", "S4", "Sn", "ccl_pwl",  
"sk38_pwl", "LWR_net", "ILWR",  
"TA", "RH", "VW", "DW",  
"ISWR", "HS_meas", "TSG",  
"sk38_pwl", "sn38_pwl", "Sd",  
"ccl_pwl", "HN24", "HN72_24",  
"HN24_7d"]
```

- ▶ All features needed for the neural network
- ▶ Some are given directly from the meteo API
- ▶ Some are given by the Alpine3D grid
- ▶ Some coefficients for the quality of the snow are missing → Snowpack

Creating a danger grid



Visualizing Danger and Snow Height in Godot



- ▶ Publish those maps in Geoserver (automated script)
- ▶ Map the danger and snow height on “gray scale”



- ▶ Framework created by the Austrian Avalanche and Torrent Service
- ▶ Solves depth-averaged mass and momentum equations for a granular flow with frictions
- ▶ Different types of avalanche : dry and wet

Avalanche types



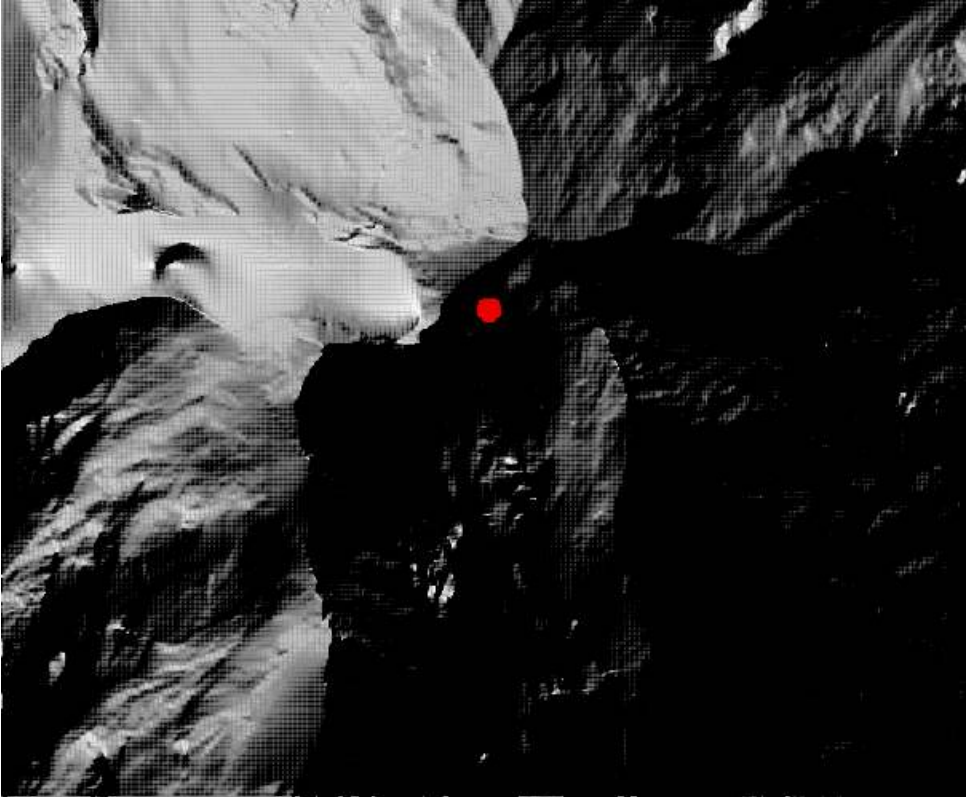
Dry avalanche



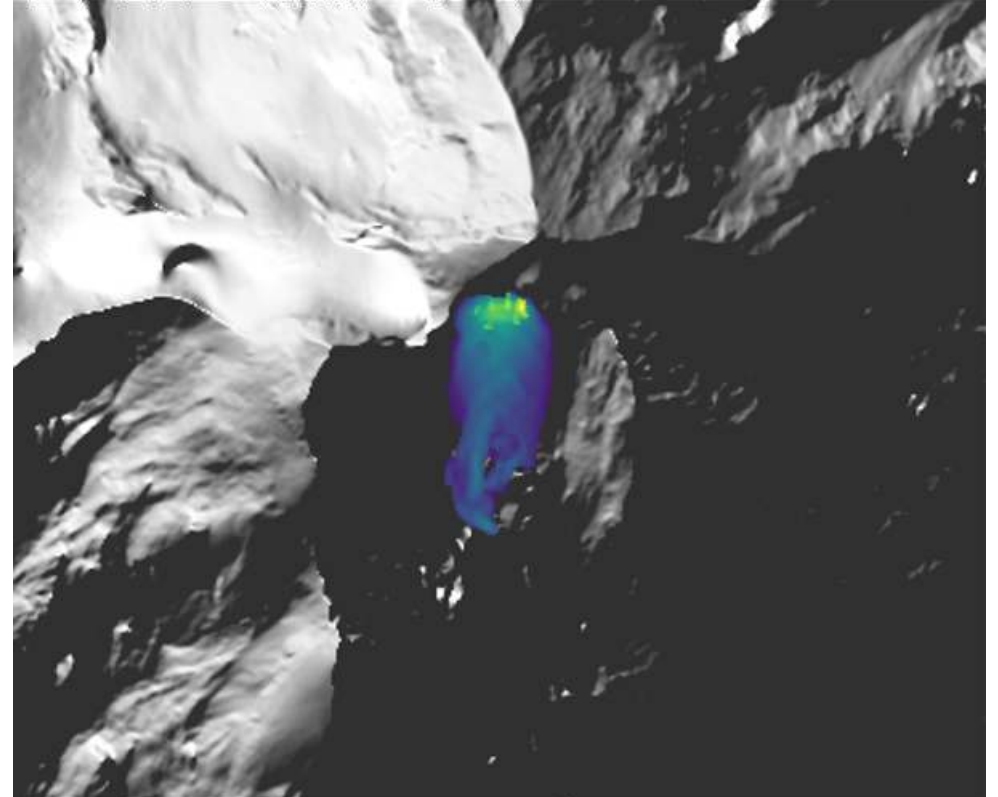
Wet avalanche

Simulation inputs and outputs

Inputs



Outputs



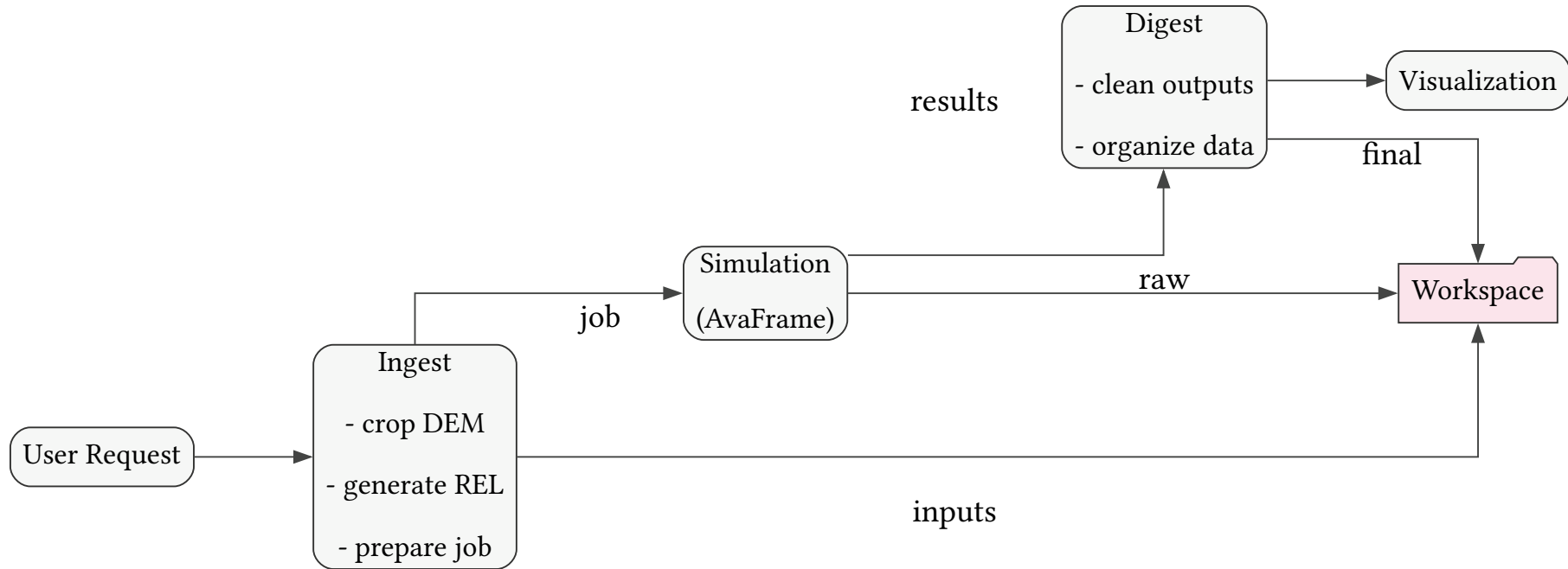


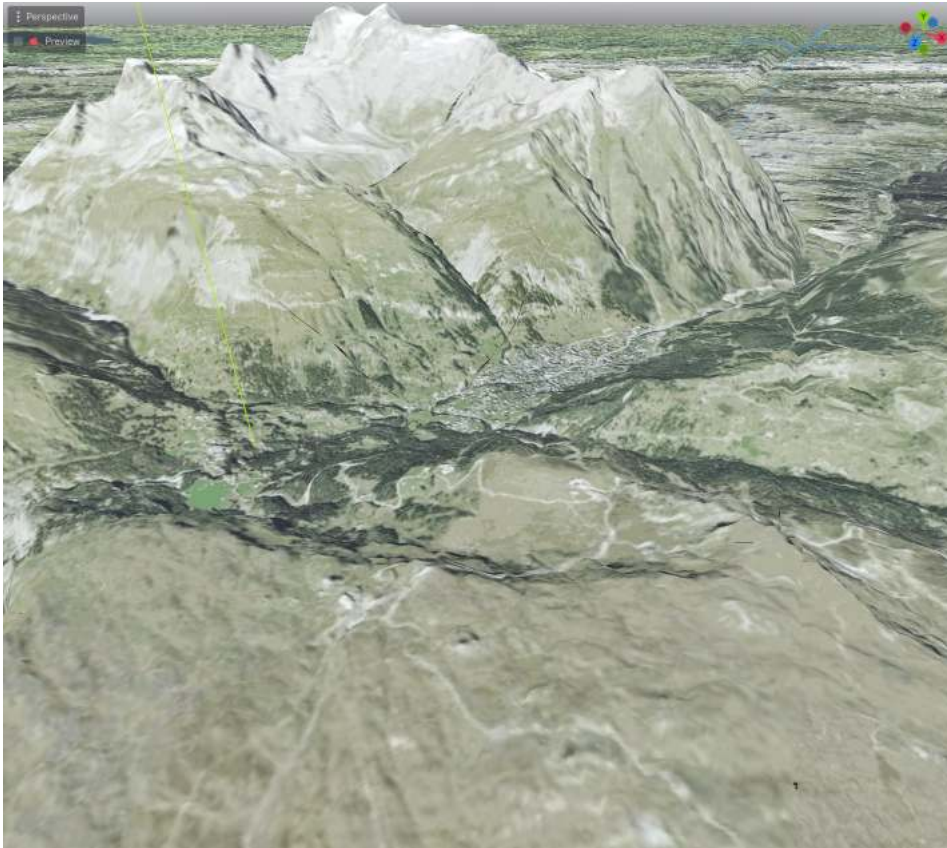
Figure 11: Simulation service workflow

Visualization



Visualization is the **main insight** our DT provides :

- ▶ visualize maps of avalanche danger level
- ▶ visualize snow depths/ various meteorological data
- ▶ visualize the path of an avalanche
- ▶ visualize endangered infrastructures depending on the path of the avalanche



We have a technical limitation here !

DEM: Digital Elevation Model

For Zermatt: 12GB



Hence, you will see a few artifacts on the edges

Figure 12: Runtime 3D visualization of Zermatt (demo later)

Technology:

- ▶ compute shaders
- ▶ raytracing
- ▶ raymarching
- ▶ dynamic terrain LODs based on WMTS

In the end, bad time investment.

The final DT is large-scale, and huge avalanche are not that common...



Figure 13: Our versatile shader approach

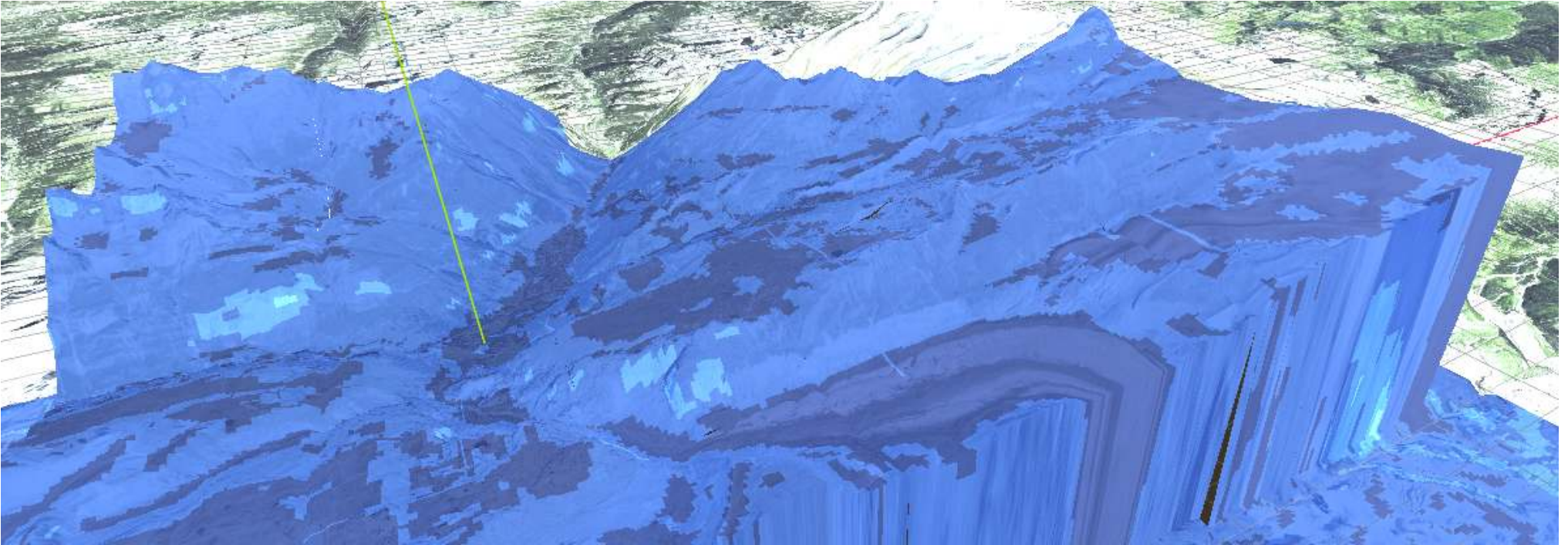


Figure 14: We can display any map

At the time being, we display snow height and avalanche danger level

Difficulties and shortcomings

- ▶ Designing the avalanche prediction logic was very tedious. SNOWPACK and Alpine3D are software made for and by weather researchers
- ▶ Working with geographic data becomes costly in resources very quickly

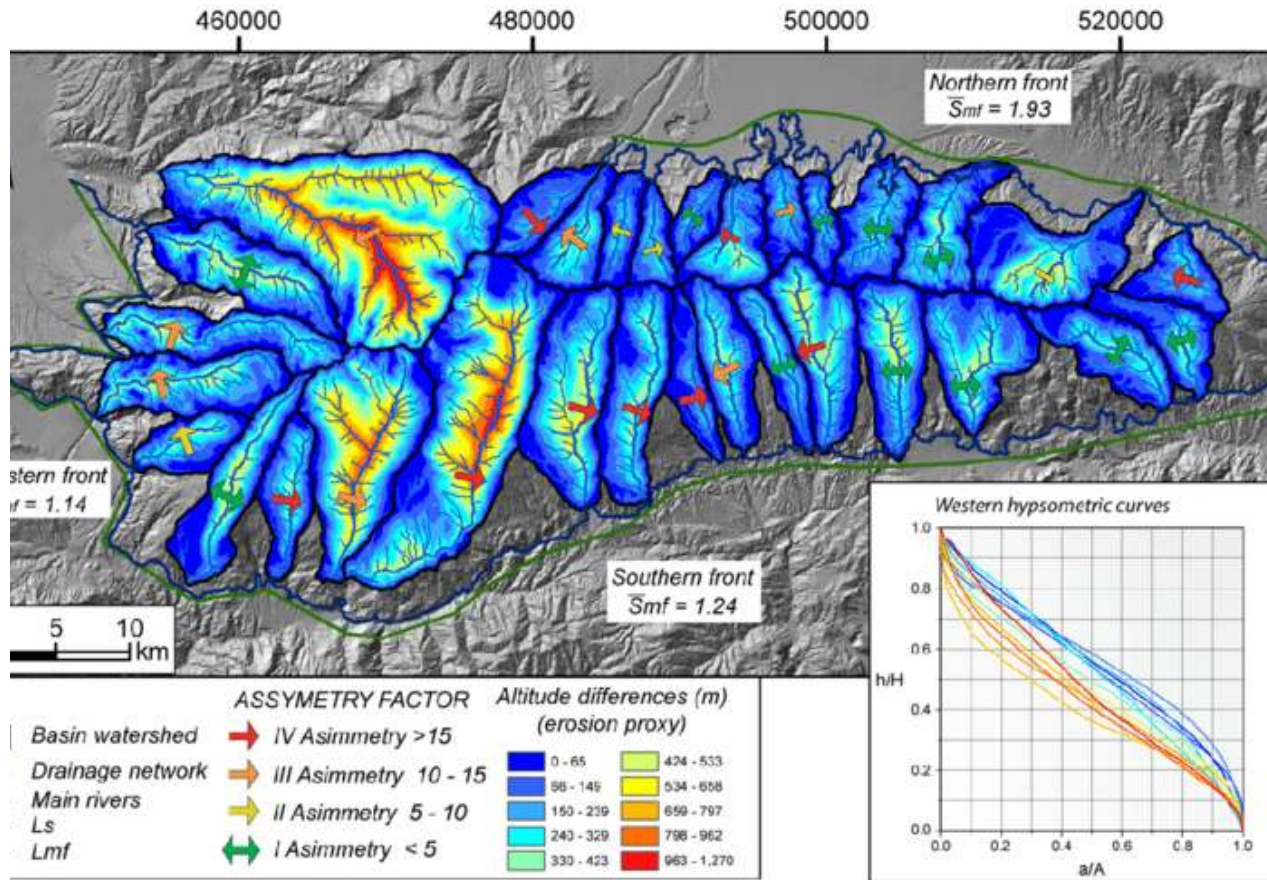
- ▶ Our vision was very large for the scale of the course
- ▶ Postponing the software integration → × respected timeline

Future work

We can **heavily** optimize most parts of the DT:

- ▶ (except) Alpine3D is slow by essence...but the its data is slow paced
- ▶ the visualization could be more efficient
- ▶ the simulation is easily scalable on MPUs and large servers (thanks OpenMP)

System of DTs !



Once the mountains are segmented, scaling the DT becomes an embarrassingly parallel problem

Figure 15: Drainage network

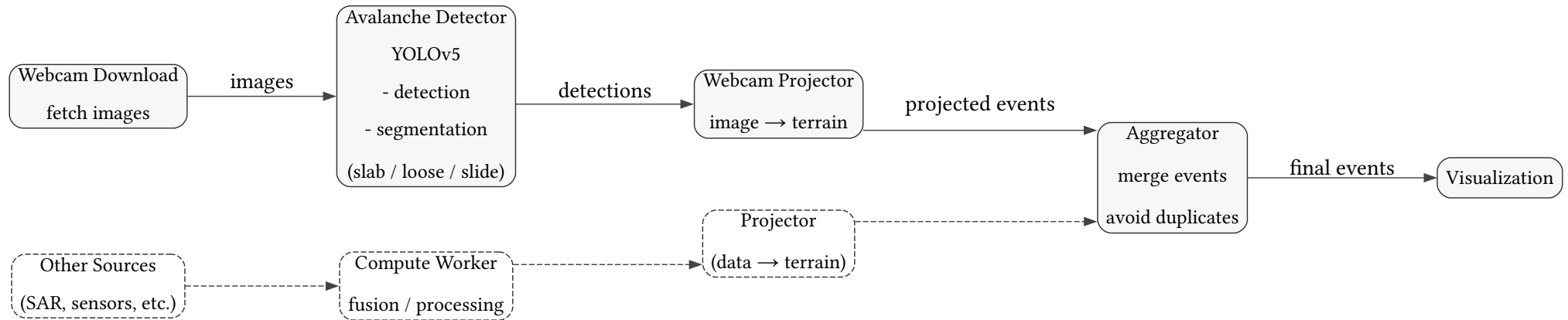


Figure 16: Webcam avalanche detector workflow

YOLOv5-based detection

(a)



(b)



A better loop...

To better close the loop, automated remote avalanche triggering is an efficient good option.